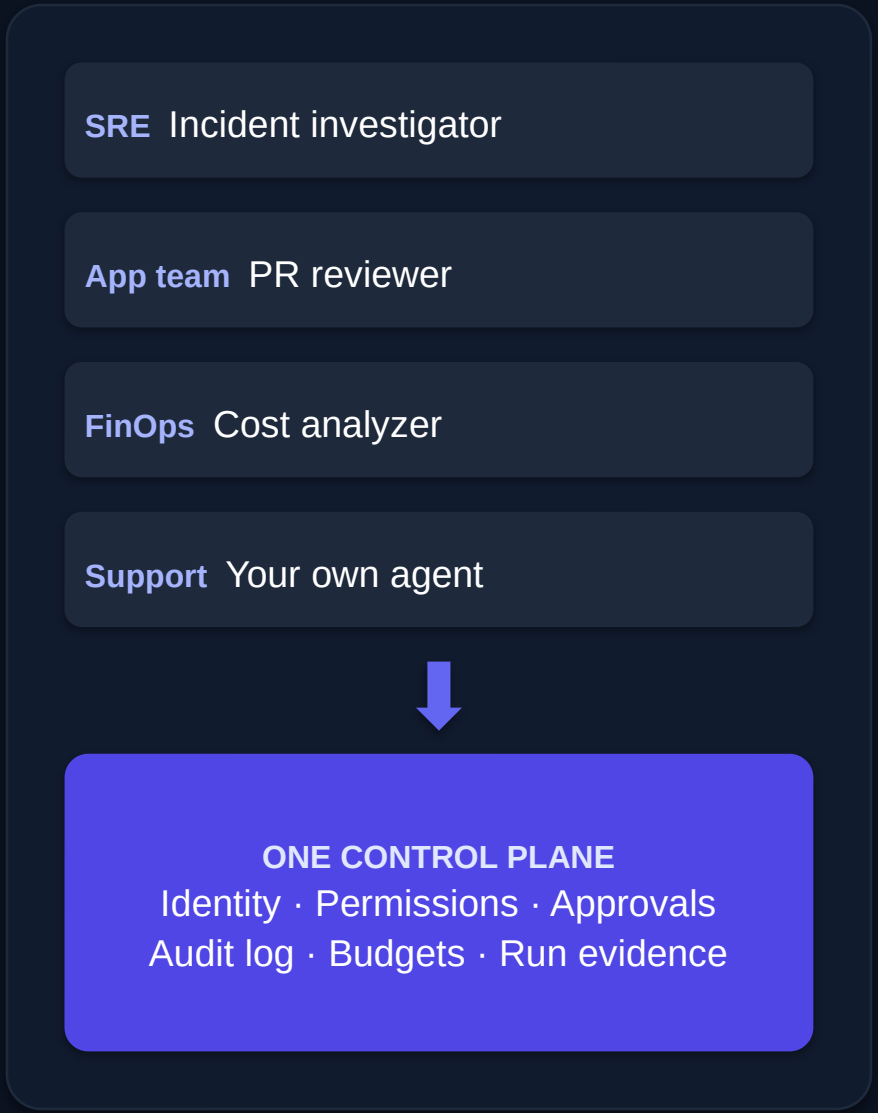
KOMODOR AGENTIC OPERATIONS PLATFORM

Your teams are already shipping agents.

Can you govern them?

Give every agent an identity, a boundary, an approval path and a record.



Echizeni Ikpah · Product Engineer

About me

Echizeni Ikpah

Product Engineer

Focus	Helping platform teams adopt AI agents safely.
For today	I built a production-like lab end to end: Kubernetes, Grafana Cloud and KAOP.
My rule	Everything you'll see, I tested on that lab first.

WHAT I CARE ABOUT

Platform teams saying yes to AI agents

without losing control of what those agents can touch, or the record of what they did.

What to expect

About 30 minutes, then your questions.

5 min	**Your challenges**	Where agent adoption hurts today.
10 min	**How KAOP addresses them**	Operating model, governance, visibility.
12 min	**Live: one real incident**	From alert to reviewed fix.
3 min	**A first step**	Shaped around your team.
15 min	**Your questions**	Anything, including the hard ones.

YOU WILL SEE, LIVE

- ✓ A real alert from Grafana Cloud
- ✓ Two agents investigating at once
- ✓ A person approving the fix
- ✓ An AI review of a real pull request
- ✓ The full record of every step

Agents are spreading faster than your operating model

Each team's agent makes sense on its own. Nobody manages them together.

WHO OWNS IT?

Ownership

Agents appear team by team. Nobody has the full list.

WHAT CAN IT TOUCH?

Access

Each agent brings its own keys. Its limits live in a prompt.

WHAT DID IT DO?

Auditability

Alert, investigation and fix sit in three different tools.

40%+ **of agentic AI projects will be canceled by the end of 2027.**

Escalating costs, unclear business value or inadequate risk controls. Gartner, Inc., cited in Komodor's KAOP launch.

QUESTION Ownership, access or auditability: which hurts your team most today?

KAOP runs agents like production services

The context, memory and governance Komodor built for its own agents, ready for yours from day one.

BUILD

Start ready, or bring your own

Pre-built workflows for AI SRE, software ops and cost. 50+ Komodor agents. Or import yours.

RUN

Real triggers, real workflows

Alerts, schedules and Slack start runs. Workflows route to specialists.

GOVERN

Enforced, not instructed

Identity, default-deny permissions, guardrails, approvals and audit.

OPTIMIZE

Keep what earns its cost

Every run's evidence and cost. Evaluations and budgets.

KOMODOR'S AI SRE IN PRODUCTION WITH NEBIUS DELL CISCO PRUDENTIAL

A prompt is not a permission model

KAOP governs an agent the way it governs a person.

	LAYER	ASKS	STOPS
1	**Identity**	Who is acting?	Unaccountable actions
2	**Permissions**	Allowed on this?	Reaching beyond the job
3	**Standards**	Built to our rules?	Agents with no owner or budget
4	**Guardrails**	Safe right now?	Destructive tool calls
5	**Approvals**	Should a person decide?	Irreversible actions alone
6	**Audit log**	What happened?	Changes nobody can explain

IN TODAY'S LAB

Investigators
Read only. Kubernetes and the gateway enforce it.

Remediation
Proposes fixes. Can't apply them.

Code reviewer
Comments only. Can't approve or merge.

PRINCIPLE Enforced on every run. Prompts can drift; enforcement doesn't.

One record from the alert to the merged fix

What the platform team sees, without asking anyone.

Fleet health

Every agent, its owner, its status.

Runs & evidence

Inputs, tool calls, reasoning, output, cost.

Attention

Decisions waiting on a person.

Audit log

Who changed what, and when.

Spend

Cost per agent and team, with budgets.

Access explorer

What can this agent actually do?

RESULT Inventory, access and audit become a screen, not a spreadsheet.

One incident, two governed workflows

Built from KAOP catalog agents. No custom code.

WORKFLOW 1 · INCIDENT · owned by SRE · starts on a Grafana alert



WORKFLOW 2 · CODE REVIEW · owned by the app team · starts on any pull request



NOTICE Agents investigate, propose and review. People approve and change production.

Who can do what, and what enforces it

Least privilege per agent, enforced outside the agent.

Agent	Can read	Can change	Enforced by
Kubernetes RCA	Pods, events, logs, workloads	Nothing	Kubernetes RBAC on its own identity
Grafana investigator	Metrics, logs, alerts	Nothing	KAOP gateway blocks write tools
Orchestrator	Findings	Nothing	May call only allowlisted agents
Remediation	The investigation	Nothing	Plan-only by design
Code reviewer	The PR diff	One comment	GitHub app scopes

VERIFIED ON THIS CLUSTER

KAOP's agent permissions checked with kubectl: reads pods, events, logs. Denied secrets, exec, delete, patch, scale.

Live: one bad change, one Redis outage

Real cluster, real alert, real agents.

BREAK

1
Bad change
Liveness checks Redis.

2
Redis goes down
Started when we began.

3
Amplified
Kubernetes kills the healthy API.

INVESTIGATE

4
Detect
Grafana alerts KAOP.

5
Investigate
Root cause, confidence, evidence.

DECIDE AND FIX

6
Decide
I approve, then apply.

7
Prevent
One-line PR, reviewed by an agent.

MEASURED ON THIS LAB

Bad check API killed 5 times in 5 minutes, alert fired

Fixed check 0 kills, alert stayed quiet

WATCH FOR Redis is the trigger. The health check is the root cause.

Autonomy, earned one step at a time

From human in the loop to human on the loop. Each step is a permission you grant.

1 TODAY

Investigate

Root cause, confidence and evidence, minutes after the alert.

2 TODAY

Propose

The fix, with the command, an undo plan and a check that it worked.

3 NEXT

Approve to execute

A person approves; an agent applies it under its own identity.

4 LATER

Pre-approved actions

Low-risk, reversible fixes run inside guardrails. People review the record.

GOAL Lower time to resolution, fewer 3 a.m. pages, engineers back on the roadmap.

Let's shape the first step together

Built around your team, at your pace.

ALIGN

A scoping session

Agree the workflow, the owner, how we measure success, and the guardrails.

PROVE

One workflow

One cluster. Read-only. Every change approved by a person.

EXPAND

At your pace

More workflows, teams and autonomy, once the results hold.

FROM YOUR TEAM

- A driver who owns it day to day
- The budget owner, who decides when to expand
- Two or three dates for the scoping session

FROM KOMODOR

- Help with setup and integrations
- Catalog agents, ready to run
- A joint review of the results

NEXT Pick a date for the scoping session with your driver and budget owner.

Thank you

Three things to remember

1. Agents need identities and permissions, not just prompts.
2. Every action needs a boundary and a record.
3. Autonomy is earned. Start with one workflow and one owner.

NEXT STEP

A scoping session with your driver and budget owner.

Echizeni Ikpah
Product Engineer

Questions?

Appendix

Reference material

A Sources for this presentation

B Common questions

C Measured results

Sources for this presentation

Every claim traces to Komodor's material or to this lab.

Komodor website and KAOP launch	Positioning: build, run, govern, optimize. The Gartner figure.
KAOP docs: security and governance	The six layers, approvals, audit log, Access explorer.
KAOP docs: agent catalog	What each agent reads, changes, and where it stops.
KAOP docs: incident module	The alert-to-remediation workflow.
This lab	A real fault, a real alert, permissions verified, fix measured.

APPROACH Start from your challenges, prove it on a real incident, then agree the first step together.

Common questions

“We'll build this ourselves.”	You can build an agent. The hard part is identity, approvals, audit, budgets and evaluation around every agent, for every team. That's the platform.
“Komodor already has Klaudia. Why KAOP?”	Klaudia is Komodor's AI SRE. KAOP is the control center for all your agents: Klaudia-based, Komodor's and your own.
“What if the agent is wrong?”	It separates what it saw from what it guessed, every change needs a person, and KAOP grades runs over time.
“How long does onboarding take?”	Catalog agents deploy in minutes. The real time goes into agreeing the workflow, the owner and the guardrails, which is why I'd start with a scoping session.
“What does it cost?”	Every run shows its cost, and budgets block new runs at a limit. We'd agree a budget in the scoping session and track cost per useful investigation.

Measured results

The same Redis outage, measured both ways.

	Liveness checks Redis (bad)	Liveness checks the process (fix)
Times Kubernetes killed the healthy API	5 in a 5-minute outage	0
Liveness failures	Yes	None
Grafana restart alert	Fired at +3 minutes	Stayed Normal
API back after Redis returned	4.8 s to 3.6 min (restart back-off)	6.5 s

Measured on 2026-10-24 and 2026-10-25 on a single-node kind cluster. With the bad check, recovery waits on Kubernetes' restart back-off (10 s, doubling up to 5 minutes), so it depends on how long the outage lasted.